Video Recording

Capture browser automation sessions as video for debugging, documentation, or verification. Produces WebM (VP8/VP9 codec).

Basic Recording

```
# Open browser first
playwright-cli open

# Start recording
playwright-cli video-start demo.webm

# Add a chapter marker for section transitions
playwright-cli video-chapter "Getting Started" --description="Opening the homepa

# Navigate and perform actions
playwright-cli goto https://example.com
playwright-cli snapshot
playwright-cli click e1

# Add another chapter
playwright-cli video-chapter "Filling Form" --description="Entering test data" -
playwright-cli fill e2 "test input"

# Stop and save
playwright-cli video-stop
```

Best Practices

1. Use Descriptive Filenames

```
# Include context in filename
playwright-cli video-start recordings/login-flow-2024-01-15.webm
playwright-cli video-start recordings/checkout-test-run-42.webm
```

2. Record entire hero scripts.

When recording a video for the user or as a proof of work, it is best to create a code snippet and execute it with run-code. It allows pulling appropriate pauses between the actions and annotating the video. There are new Playwright APIs for that.

1) Perform scenario using CLI and take note of all locators and actions. You'll need those locators to request their bounding boxes for highlight.

2) Create a file with the intended script for video (below). Use pressSequentially w/ delay for nice typing, make reasonable pauses.
3) Use playwright-cli run-code –filename your-script.js

Important: Overlays are `pointer-events: none` — they do not interfere with page interactions. You can safely keep sticky overlays visible while clicking, filling, or performing any actions on the page.

```
async page => {
  await page.screencast.start({ path: 'video.webm', size: { width: 1280, height:
  await page.goto('https://demo.playwright.dev/todomvc');

  // Show a chapter card — blurs the page and shows a dialog.
  // Blocks until duration expires, then auto-removes.
  // Use this for simple use cases, but always feel free to hand-craft your own
  // overlay via await page.screencast.showOverlay().
  await page.screencast.showChapter('Adding Todo Items', {
    description: 'We will add several items to the todo list.',
    duration: 2000,
  });

  // Perform action
  await page.getByRole('textbox', { name: 'What needs to be done?' }).pressSeque
  await page.getByRole('textbox', { name: 'What needs to be done?' }).press('Ent
  await page.waitForTimeout(1000);

  // Show next chapter
  await page.screencast.showChapter('Verifying Results', {
    description: 'Checking the item appeared in the list.',
    duration: 2000,
  });

  // Add a sticky annotation that stays while you perform actions.
  // Overlays are pointer-events: none, so they won't block clicks.
  const annotation = await page.screencast.showOverlay(`
    <div style="position: absolute; top: 8px; right: 8px;
      padding: 6px 12px; background: rgba(0,0,0,0.7);
      border-radius: 8px; font-size: 13px; color: white;">
      ✓ Item added successfully
    </div>
  `);

  // Perform more actions while the annotation is visible
  await page.getByRole('textbox', { name: 'What needs to be done?' }).pressSeque
  await page.getByRole('textbox', { name: 'What needs to be done?' }).press('Ent
  await page.waitForTimeout(1500);
```

```
  // Remove the annotation when done
  await annotation.dispose();

  // You can also highlight relevant locators and provide contextual annotations
  const bounds = await page.getByText('Walk the dog').boundingBox();
  await page.screencast.showOverlay(`
    <div style="position: absolute;
      top: ${bounds.y}px;
      left: ${bounds.x}px;
      width: ${bounds.width}px;
      height: ${bounds.height}px;
      border: 1px solid red;">
    </div>
    <div style="position: absolute;
      top: ${bounds.y + bounds.height + 5}px;
      left: ${bounds.x + bounds.width / 2}px;
      transform: translateX(-50%);
      padding: 6px;
      background: #808080;
      border-radius: 10px;
      font-size: 14px;
      color: white;">Check it out, it is right above this text
    </div>
  `, { duration: 2000 });

  await page.screencast.stop();
}
```

Embrace creativity, overlays are powerful.

Overlay API Summary

Method	Use Case
`page.screencast.showChapter(title, { description?, duration?, styleSheet? })`	Full-screen chapter card with blurred backdrop — ideal for section transitions
`page.screencast.showOverlay(html, { duration? })`	Custom HTML overlay — use for callouts, labels, highlights
`disposable.dispose()`	Remove a sticky overlay added without duration

Method	Use Case
`page.screencast.hideOverlays()` / `page.screencast.showOverlays()`	Temporarily hide/show all overlays

Tracing vs Video

Feature	Video	Tracing
Output	WebM file	Trace file (viewable in Trace Viewer)
Shows	Visual recording	DOM snapshots, network, console, actions
Use case	Demos, documentation	Debugging, analysis
Size	Larger	Smaller

Limitations

- Recording adds slight overhead to automation
- Large recordings can consume significant disk space